Design and Implementation of an Advanced Smart Parking System with Owner Control and Intelligent Access Management

ADEL HUSHAM MOHAMEDAIN YOUSUF[1]
[1]231090057-1

[1] FAKULTI KEJURUTERAAN TEKNOLOGI
ELEKTRIK (FKTE) - UNIVERSITI MALAYSIA PERLIS (UNIMAP), MALAYSIA
[1]s231090057-1@studentmail.unimap.ed.my

Abstract— This project presents the design and implementation of an Advanced Smart Parking System using the Raspberry Pi Pico W microcontroller. The system was first developed and tested using the Wokwi simulation platform and later implemented using real hardware components. The main objective of the project is to improve parking management by providing secure access control, parking slot monitoring, and real-time status indication [1].

The system includes a password-based entry mechanism using a 4x3 keypad encoder, parking slot status LEDs, a 16x2 LCD display, and a dual 7-segment display for showing the number of available parking spaces. In the simulation stage, servo motors and a buzzer were used to represent gate control and alarm functions. During hardware implementation, these components were replaced with LEDs due to component availability limitations in the laboratory environment. The LEDs successfully simulated the same operational behavior of gate opening and alarm indication.

The system was evaluated through both simulation and hardware testing under different operating conditions such as correct password entry, wrong password attempts, parking slot selection, vehicle exit operation, and emergency alarm activation. The evaluation process confirmed that the system operated correctly in both environments and provided reliable real-time feedback to the user.

The final results demonstrate that the proposed smart parking system can provide an effective low-cost solution for parking access management and slot monitoring. The project also shows the importance of integrating simulation tools with hardware implementation in embedded systems development [2].

I. INTRODUCTION

Smart parking systems are becoming increasingly important in modern cities and buildings because they help reduce traffic congestion, improve parking management, and save time for drivers searching for available parking spaces [1]. Traditional parking systems usually depend on manual monitoring and do not provide real-time information about parking availability. As a result, drivers may experience delays, confusion, and inefficient parking operations. Smart parking technology solves these problems by combining sensors, displays, automated access control, and embedded systems technologies [2].

This project presents the design and implementation of an Advanced Smart Parking System using the Raspberry Pi Pico W microcontroller. The system was first designed and tested using the Wokwi simulation platform and later implemented using real hardware components in the laboratory environment. The project combines several embedded system concepts including keypad interfacing, password-based authentication, LCD communication, 7-segment display control, parking slot monitoring, and security alarm operation.

The main objective of this project is to develop a secure and user-friendly parking management system capable of monitoring parking slot availability and controlling vehicle entry and exit operations automatically. The project also aims to improve parking security by implementing a password verification system and emergency alarm mode. The measurable objectives of the project are:

- To design and simulate a smart parking system using the Wokwi simulation environment.
- To implement the parking system using Raspberry Pi Pico W hardware components.
- To display real-time parking slot availability using LEDs, LCD, and dual 7-segment displays.
- To provide secure parking access using a password-protected keypad system.
- To evaluate and compare the system performance in both simulation and hardware implementation environments.

The developed system contains three parking slots. Each parking slot is represented using an LED indicator to show whether the slot is occupied or available. Additional status LEDs are used to indicate system conditions

such as parking availability and parking full status. A 16x2 LCD display is used to provide system messages including password requests, slot selection information, parking conditions, and emergency alerts. A dual 7-segment display continuously shows the number of available parking spaces in real time.

The parking system operates using a password-based authentication mechanism. Initially, the system remains locked and requests the user to enter a password using the 4x4 keypad encoder. If the correct password is entered, the parking system becomes active and the current parking slot status is displayed on the LCD. The user can then select one of the available parking slots. If the selected slot is available, the entry process is activated automatically. In the Wokwi simulation environment, servo motors and a buzzer were used to simulate gate opening and alarm operations. However, during hardware implementation, these components were replaced with LEDs due to component availability limitations in the laboratory environment. The LEDs successfully simulated the same operational behavior of the gate control and alarm indication systems.

The project also includes a security protection mechanism. If an incorrect password is entered three consecutive times, the system enters temporary lock mode and activates an alarm indication with a 10-second countdown displayed on the dual 7-segment display. In addition, an emergency alarm push button and a vehicle exit button are included to simulate real parking system operations.

Figure 1 shows the overall block diagram of the proposed Smart Parking System. The Raspberry Pi Pico W acts as the main controller and communicates with all input and output components. The keypad encoder is used for password entry and parking slot selection, while the LCD and dual 7-segment displays provide real-time system information. Parking slot LEDs indicate slot occupancy status, and additional indicators are used for system availability and alarm conditions. In the Wokwi simulation, servo motors and a buzzer were used for gate and alarm operations, while in the hardware implementation these components were replaced by LEDs to simulate the same functionality.

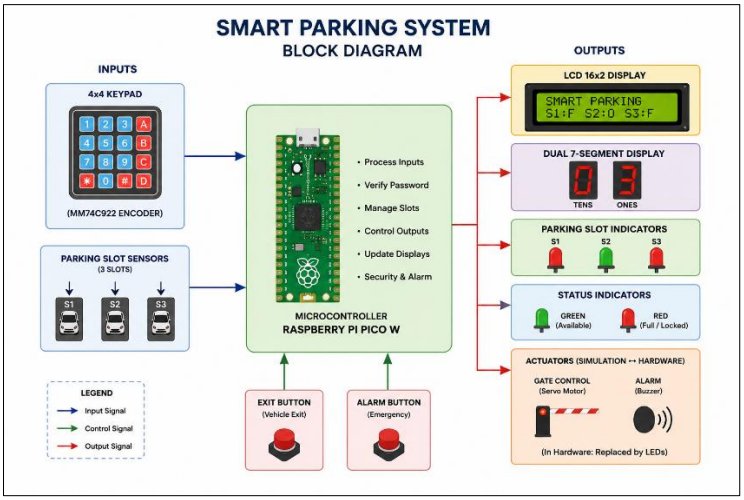

Figure 1: Block Diagram of the Proposed Smart Parking System.

The system operation begins when the user powers ON the system. The LCD initially displays a locked parking message requesting the user password. After successful password verification, the parking system becomes active and continuously updates the LCD, parking LEDs, and dual 7-segment display according to parking slot availability and system conditions. The system provides a simple and efficient smart parking solution suitable for embedded systems applications and educational IoT-based projects [3].

II. METHODOLOGY

The proposed Smart Parking System was developed using both Wokwi simulation and real hardware implementation. The project was designed to monitor parking slot availability, control parking access using password authentication, and provide real-time parking status indication using LEDs, LCD display, and dual 7-segment displays [1].

The system was first tested using the Wokwi simulation platform to verify the operational logic and component interaction before hardware implementation. The simulation included the Raspberry Pi Pico W, 4×4 keypad, LCD display, CD4511 decoder, dual 7-segment display, parking LEDs, servo motors, buzzer, and push buttons. In the simulation environment, 220Ω resistors were used for LEDs, while 330Ω resistors were connected to the CD4511 decoder and dual 7-segment display circuit.

Figure 2 shows the Wokwi simulation setup of the proposed Smart Parking System.

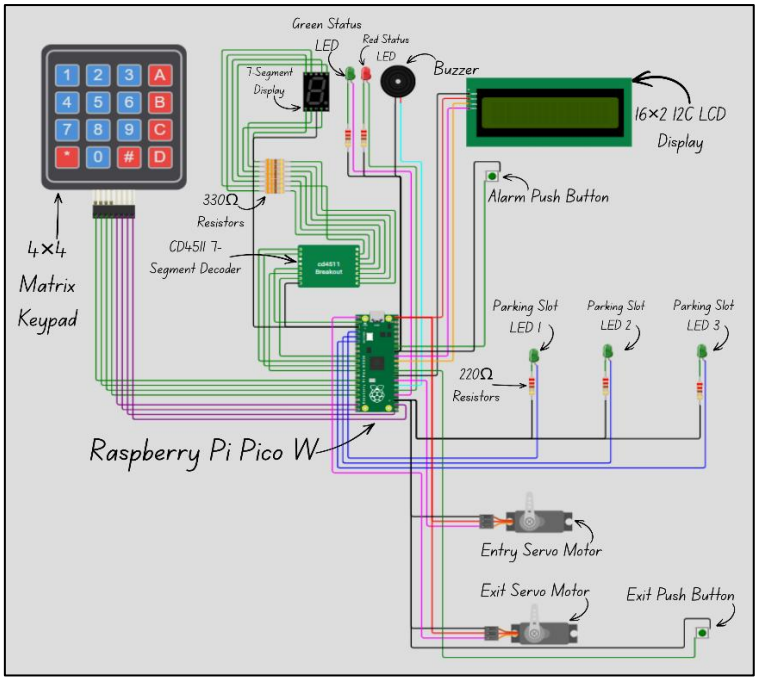

Figure 2: Wokwi Simulation Setup of the Smart Parking System.

After successful simulation testing, the system was implemented using real hardware components in the laboratory environment. The hardware setup included the Raspberry Pi Pico W, MM74C922 keypad encoder, LCD display, dual 7-segment display, parking LEDs, status LEDs, and push buttons. During hardware implementation, the servo motors and buzzer used in the simulation were replaced with LEDs due to component availability limitations. The LEDs were used to simulate gate operation and alarm indication while maintaining the same system logic [2].

Figure 3 shows the hardware implementation of the proposed Smart Parking System.

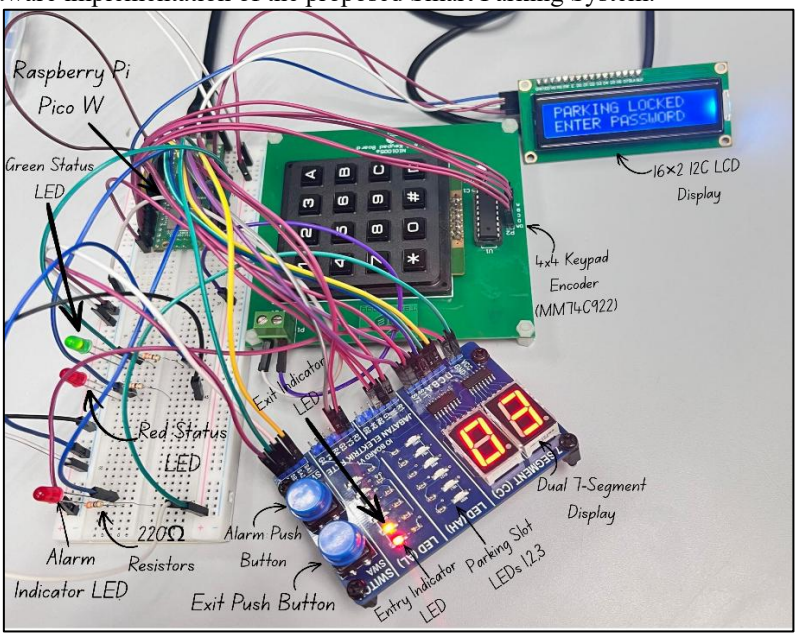

Figure 3: Hardware Implementation of the Smart Parking System.

Several testing procedures were performed during both simulation and hardware implementation stages. The system was tested for password authentication, parking slot selection, exit operation, emergency alarm activation, LCD functionality, and 7-segment display operation. The evaluation process confirmed stable operation and correct interaction between all system components.

Figure 4 illustrates the operational logic flow of the proposed Smart Parking System. The flowchart explains the complete system sequence, including password verification, parking slot monitoring, entry and exit operations, alarm activation, and temporary system lock mode.

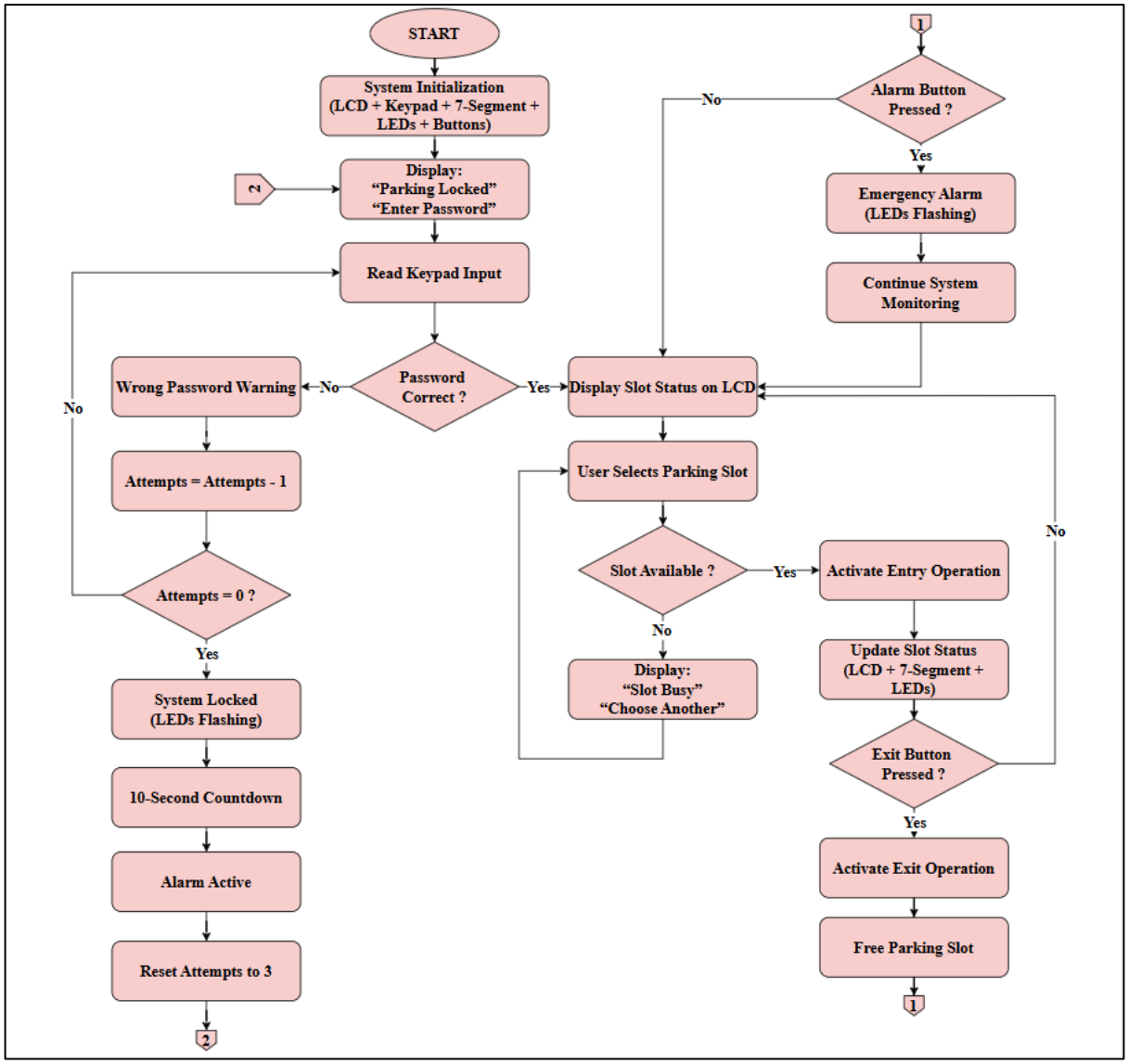

Figure 4: Operational Flowchart of the Proposed Smart Parking System.

The Raspberry Pi Pico W acts as the main controller and communicates with all input and output components. The keypad encoder is connected using digital input pins, while the LCD display communicates through the I2C interface. The dual 7-segment display operates using BCD output signals connected to the decoder circuit.

Table 1: presents the main hardware pin connections used in the proposed system.

Component	Raspberry Pi Pico W Pins	Configuration
MM74C922 Keypad Encoder	GP10 – GP14	Digital Input
LCD 16×2 I2C	GP26, GP27	I2C Communication
Dual 7-Segment Display	GP6 – GP9	BCD Output
Parking Slot LEDs	GP2, GP3, GP4	Digital Output
Green Status LED	GP18	Digital Output
Red Status LED	GP19	Digital Output
Alarm Indicator LED	GP20	Digital Output
Exit Push Button	GP22	Pull-Up Input
Alarm Push Button	GP28	Pull-Up Input

III. RESULT

The proposed Smart Parking System was successfully tested using both Wokwi simulation and real hardware implementation. The system achieved stable operation for password authentication, parking slot monitoring, LCD communication, alarm activation, and parking slot counting functions.

Figure 5 shows the hardware startup condition and password authentication process. The LCD display successfully requested the user password before allowing access to the parking system. The red status LED remained active while the parking system was locked.

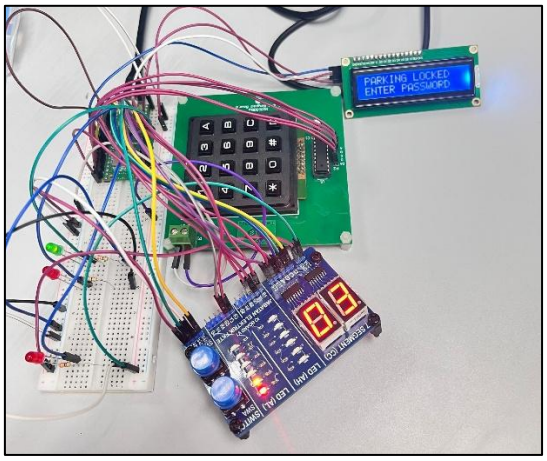

Figure 5: Hardware Startup and Password Authentication Mode.

Figure 6 shows the parking slot monitoring operation after successful password verification. The LCD display correctly presented the status of all parking slots, while the dual 7-segment display updated the number of available parking spaces in real time. The green status LED indicated that parking spaces were available.

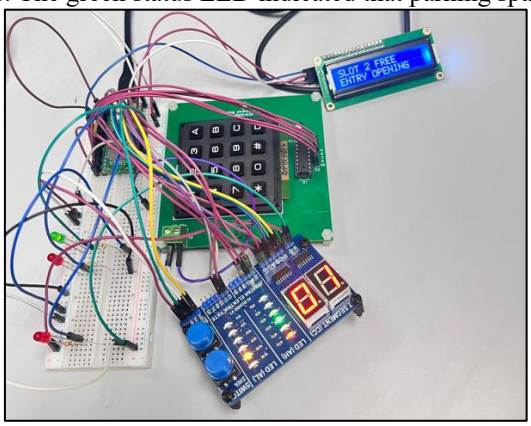

Figure 6: Hardware Parking Slot Monitoring Operation.

Figure 7 shows the emergency alarm condition during the Wokwi simulation process. The buzzer and warning LEDs were activated continuously to indicate emergency alarm mode after pressing the alarm push button. The LCD display also showed the alarm warning message during system operation.

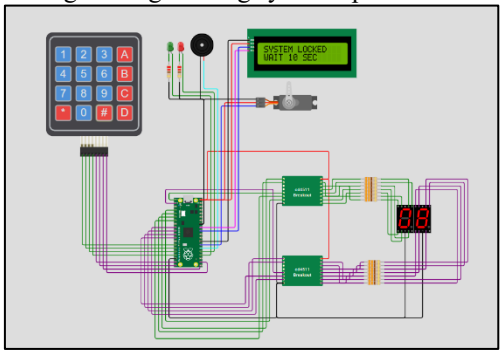

Figure 7: Wokwi Simulation Emergency Alarm Operation.

The LCD display successfully presented all required system messages, including password requests, parking slot conditions, warning notifications, and alarm indications. The keypad encoder correctly detected user input for password entry and parking slot selection. The dual 7-segment display accurately updated the number of available parking slots during both entry and exit operations.
The parking slot LEDs operated correctly according to parking availability conditions. When a parking slot became occupied, the corresponding parking LED turned ON, while free parking slots remained OFF. The green status LED indicated parking availability, whereas the red status LED indicated full parking or warning conditions.

Table 2: shows the comparison between the simulation and hardware implementations.

Function	**Simulation**	**Hardware**
Entry Gate Control	Servo Motor	Indicator LED
Exit Gate Control	Servo Motor	Indicator LED
Alarm System	Buzzer	Flashing LED
LCD Display	Functional	Functional
Password Authentication	Successful	Successful
Parking Counter	Functional	Functional

Table 3: presents the comparison between expected and actual system behavior.

Test Scenario	**Expected Result**	**Actual Result**	**Status**
Correct Password Entry	System Unlocks	Operated Correctly	PASS
Wrong Password Attempts	Temporary Lock Activated	Operated Correctly	PASS
Parking Slot Selection	Slot Status Updated	Operated Correctly	PASS
Exit Operation	Parking Slot Released	Operated Correctly	PASS
Alarm Activation	LEDs Flashing	Operated Correctly	PASS
LCD Message Display	Correct Messages	Operated Correctly	PASS
7-Segment Counter	Real-Time Update	Operated Correctly	PASS

Table 4: presents the system response during different parking and alarm operation scenarios.

Input Scenario	**LCD Output**	**7-Segment Display**	**LED Indicator Status**
Correct password entered	“Parking Open”	3	Green LED ON
Wrong password entered	“Wrong Password”	3	Red LED Flashing
Three wrong attempts	“System Locked”	Countdown 10→0	All LEDs Flashing
Slot 1 selected and free	“Slot 1 Free”	2	Slot 1 LED ON
Slot 2 selected and free	“Slot 2 Free”	1	Slot 2 LED ON
Slot 3 selected and free	“Slot 3 Free”	0	Slot 3 LED ON
Parking full	“Parking Full”	0	Red Status LED ON
Exit button pressed	“Car Exited”	Updated +1	Exit LED ON
Alarm button pressed	“Emergency Alarm”	Flashing	Alarm LED Flashing

The results obtained from both simulation and hardware testing confirm that the proposed Smart Parking System successfully performed parking monitoring, password authentication, parking slot counting, and emergency alarm operations. The LCD display, LEDs, and dual 7-segment display updated correctly according to the system conditions. The system also demonstrated stable real-time operation during both simulation and hardware implementation stages [1], [2], [3].

IV. DISCUSSION

The results obtained show that the proposed Smart Parking System operated successfully in both Wokwi simulation and real hardware implementation. The system correctly performed password authentication, parking slot monitoring, LCD communication, parking slot counting, and emergency alarm indication functions [3].

The simulation environment behaved as intended and helped verify the operational logic and component interaction before hardware implementation. The Wokwi simulation also simplified debugging and software testing during the development stage.
The hardware implementation produced results similar to the simulation model. The keypad encoder successfully detected user input, while the LCD display correctly showed parking status messages and warning notifications. The dual 7-segment display accurately updated the number of available parking slots during parking entry and exit operations.
Some differences existed between the simulation and hardware implementations. In the simulation environment, servo motors and a buzzer were used for gate control and alarm indication. However, in the hardware implementation, these components were replaced with LEDs due to component availability limitations in the

laboratory. Despite this modification, the system maintained the same operational logic because the LEDs successfully simulated gate operation and alarm activation behavior.
Minor hardware issues such as keypad mapping differences and 7-segment display blinking were encountered during implementation. These problems were solved through software debugging and pin configuration adjustments until stable system operation was achieved.
Overall, the evaluation confirmed that the proposed Smart Parking System successfully achieved the project objectives and demonstrated reliable operation in both simulation and hardware environments [1], [2], [3].

V. CONCLUSIONS

The proposed Smart Parking System was successfully designed, simulated, and implemented using the Raspberry Pi Pico W microcontroller platform. The system achieved the main project objectives, including password-based parking access control, parking slot monitoring, real-time parking availability display, emergency alarm indication, and parking slot counting using a dual 7-segment display [1].
The Wokwi simulation environment provided successful verification of the system logic and component interaction before hardware implementation. The simulation stage helped validate keypad operation, LCD communication, parking slot management, alarm activation, and system monitoring functions.
The hardware implementation also demonstrated stable and reliable operation. The LCD display correctly presented system messages, the keypad encoder accurately detected user input, and the dual 7-segment display successfully updated parking slot availability in real time. Although servo motors and the buzzer used in the simulation were replaced with LEDs in the hardware prototype, the overall operational behavior and system functionality remained consistent with the original simulation design.
The comparison between simulation and hardware results confirmed that the proposed system performed as intended with only minor hardware-related adjustments during implementation. The project successfully demonstrated the practical application of embedded systems, digital interfacing, monitoring, and automation techniques in smart parking management systems [2], [3].
Overall, the proposed Smart Parking System achieved reliable operation and met the required functional objectives in both simulation and hardware environments.

REFERENCES

[1] Raspberry Pi Ltd., “Raspberry Pi Pico W Datasheet,” Raspberry Pi Documentation, 2023. [Online]. Available: https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf
[2] Wokwi, “Wokwi Online Electronics Simulator,” 2024. [Online]. Available: https://wokwi.com/
[3] Adafruit Industries, “CircuitPython Documentation,” 2024. [Online]. Available: https://docs.circuitpython.org/
[4] Texas Instruments, “CD4511B CMOS BCD-to-7-Segment Latch Decoder Drivers,” Texas Instruments Datasheet, 2021. [Online]. Available: https://www.ti.com/lit/ds/symlink/cd4511b.pdf
[5] Hitachi, “HD44780U LCD Controller/Driver Datasheet,” Hitachi Semiconductor, 1998. [Online]. Available: https://www.sparkfun.com/datasheets/LCD/HD44780.pdf
[6] NKK Switches, “MM74C922 16-Key Encoder Datasheet,” Fairchild Semiconductor, 2000. [Online]. Available: https://www.alldatasheet.com/datasheet-pdf/pdf/53731/FAIRCHILD/MM74C922.html
[7] S. Monk, *Programming Raspberry Pi Pico with Python*, New York, NY, USA: McGraw-Hill Education, 2021.
[8] J. M. Hughes, *MicroPython for the Internet of Things*, Sebastopol, CA, USA: O’Reilly Media, 2021.
[9] M. Margolis, B. Jepson, and N. R. Weldin, *Arduino Cookbook*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2020.
[10] A. Badamasi, “The Working Principle of Microcontroller and Embedded Systems,” in *Proc. IEEE Int. Conf. Electronics, Computer and Computation*, Abuja, Nigeria, 2014, pp. 1–4.

APPENDIX A: CIRCUIT PYTHON SOURCE CODE

Appendix A contains the complete CircuitPython source code used to implement the Smart Parking System using the Raspberry Pi Pico W microcontroller. The program includes password authentication, keypad interfacing, LCD communication, dual 7-segment display control, parking slot monitoring, LED indication, alarm operation, parking entry and exit control, and system status monitoring.
The complete CircuitPython source code is provided below.

```
import time  # Library for delay and timing
import board  # Library for Raspberry Pi Pico pins
import digitalio  # Library for digital input and output
import busio  # Library for I2C communication
import lcd  # LCD screen library
import i2c_pcf8574_interface  # I2C interface library for LCD

# ============================================================
# LCD SETUP
# ============================================================

i2c = busio.I2C(scl=board.GP27, sda=board.GP26)  # Create I2C connection

i2c_interface = i2c_pcf8574_interface.I2CPCF8574Interface(i2c, 0x27)  # LCD I2C
address

display = lcd.LCD(i2c_interface, num_rows=2, num_cols=16)  # Create LCD object

display.set_backlight(True)  # Turn on LCD light
display.set_display_enabled(True)  # Enable LCD display

# ============================================================
# 7 SEGMENT SETUP (CD4511)
# ============================================================

# BCD pins A B C D
bcd_pins = [board.GP6, board.GP7, board.GP8, board.GP9]  # BCD output pins

class CD4511:  # Class for CD4511 decoder

    def __init__(self, pins):  # Initialize function

        self.bcd = []  # Empty list for pins

        for pin in pins:  # Loop through pins

            p = digitalio.DigitalInOut(pin)  # Create digital pin
            p.direction = digitalio.Direction.OUTPUT  # Set as output

            self.bcd.append(p)  # Add pin to list

    def send_bcd(self, value):  # Send number to 7 segment

        value = max(0, min(9, value))  # Limit number from 0 to 9

        for i in range(4):  # Loop through 4 bits

            self.bcd[i].value = (value >> i) & 1  # Send bit value

segment = CD4511(bcd_pins)  # Create CD4511 object

# ============================================================
# LATCH PINS
# ============================================================

# S71 = ONES
# S72 = TENS
```

```
57.
58. s71 = digitalio.DigitalInOut(board.GP15)  # Ones digit latch
59. s72 = digitalio.DigitalInOut(board.GP16)  # Tens digit latch
60.
61. s71.direction = digitalio.Direction.OUTPUT  # Set as output
62. s72.direction = digitalio.Direction.OUTPUT  # Set as output
63.
64. s71.value = True  # Default HIGH
65. s72.value = True  # Default HIGH
66.
67. # ========================================================
68. # KEYPAD SETUP (MM74C922)
69. # ========================================================
70.
71. # DATA AVAILABLE
72. data_available = digitalio.DigitalInOut(board.GP14)  # Data available pin
73. data_available.direction = digitalio.Direction.INPUT  # Set as input
74.
75. # D0 D1 D2 D3
76. data_pins = [
77.     board.GP13,
78.     board.GP12,
79.     board.GP11,
80.     board.GP10
81. ]
82.
83. data_inputs = []  # Empty list for keypad inputs
84.
85. for pin in data_pins:  # Loop through keypad pins
86.
87.     dio = digitalio.DigitalInOut(pin)  # Create digital pin
88.     dio.direction = digitalio.Direction.INPUT  # Set as input
89.
90.     data_inputs.append(dio)  # Add pin to list
91.
92. # ========================================================
93. # KEYPAD MAP
94. # ========================================================
95.
96. keypad_map = {
97.     12: '1', 13: '2', 14: '3', 15: 'A',
98.     8: '4', 9: '5', 10: '6', 11: 'B',
99.     4: '7', 5: '8', 6: '9', 7: 'C',
100.    0: '*', 1: '0', 2: '#', 3: 'D'
101. }  # Keypad buttons map
102.
103. # ========================================================
104. # PARKING LEDS
105. # ========================================================
106.
107. slot1_led = digitalio.DigitalInOut(board.GP2)  # Slot 1 LED
108. slot1_led.direction = digitalio.Direction.OUTPUT  # Set as output
109.
110. slot2_led = digitalio.DigitalInOut(board.GP3)  # Slot 2 LED
111. slot2_led.direction = digitalio.Direction.OUTPUT  # Set as output
112.
113. slot3_led = digitalio.DigitalInOut(board.GP4)  # Slot 3 LED
114. slot3_led.direction = digitalio.Direction.OUTPUT  # Set as output
115.
116. # ========================================================
117. # STATUS LEDS
118. # ========================================================
119.
120. green_led = digitalio.DigitalInOut(board.GP18)  # Green LED
121. green_led.direction = digitalio.Direction.OUTPUT  # Set as output
122.
```

```
123. red_led = digitalio.DigitalInOut(board.GP19)  # Red LED
124. red_led.direction = digitalio.Direction.OUTPUT  # Set as output
125.
126. # ============================================================
127. # ENTRY / EXIT LAMPS
128. # ============================================================
129.
130. entry_lamp = digitalio.DigitalInOut(board.GP21)  # Entry lamp
131. entry_lamp.direction = digitalio.Direction.OUTPUT  # Set as output
132.
133. exit_lamp = digitalio.DigitalInOut(board.GP0)  # Exit lamp
134. exit_lamp.direction = digitalio.Direction.OUTPUT  # Set as output
135.
136. # ============================================================
137. # ALARM LED
138. # ============================================================
139.
140. alarm_led = digitalio.DigitalInOut(board.GP20)  # Alarm LED
141. alarm_led.direction = digitalio.Direction.OUTPUT  # Set as output
142.
143. # ============================================================
144. # BUTTONS
145. # ============================================================
146.
147. exit_button = digitalio.DigitalInOut(board.GP22)  # Exit button
148. exit_button.direction = digitalio.Direction.INPUT  # Set as input
149. exit_button.pull = digitalio.Pull.UP  # Enable pull-up resistor
150.
151. alarm_button = digitalio.DigitalInOut(board.GP28)  # Alarm button
152. alarm_button.direction = digitalio.Direction.INPUT  # Set as input
153. alarm_button.pull = digitalio.Pull.UP  # Enable pull-up resistor
154.
155. # ============================================================
156. # VARIABLES
157. # ============================================================
158.
159. password = "0571"  # System password
160.
161. user_input = ""  # User entered password
162.
163. attempts = 3  # Number of password tries
164.
165. system_open = False  # Parking state
166.
167. TOTAL_SLOTS = 3  # Total parking slots
168.
169. occupied_slots = 0  # Number of occupied slots
170.
171. parking_slots = [0, 0, 0]  # Slot status list
172.
173. # ============================================================
174. # 7 SEGMENT DISPLAY FUNCTION
175. # ============================================================
176.
177. def update_display():  # Function to update 7 segment display
178.
179.     available = TOTAL_SLOTS - occupied_slots  # Calculate free slots
180.
181.     tens = available // 10  # Tens digit
182.     ones = available % 10  # Ones digit
183.
184.     # TENS DISPLAY
185.     segment.send_bcd(tens)  # Send tens digit
186.
187.     s72.value = False  # Activate tens latch
188.     time.sleep(0.001)  # Small delay
```

```
    s72.value = True  # Disable tens latch

    # ONES DISPLAY
    segment.send_bcd(ones)  # Send ones digit

    s71.value = False  # Activate ones latch
    time.sleep(0.001)  # Small delay
    s71.value = True  # Disable ones latch

# ============================================================
# LCD FUNCTIONS
# ============================================================

def show_slots():  # Show parking slots status

    line = ""  # Empty line

    for i in range(3):  # Loop through slots

        if parking_slots[i] == 0:  # If slot is free

            line += "S" + str(i + 1) + ":F "  # Add free status

        else:  # If slot is occupied

            line += "S" + str(i + 1) + ":O "  # Add occupied status

    display.clear()  # Clear LCD

    display.set_cursor_pos(0, 0)  # First row
    display.print("SMART PARKING")  # Print title

    display.set_cursor_pos(1, 0)  # Second row
    display.print(line[:16])  # Print slots status

def show_message(line1, line2):  # Show message on LCD

    display.clear()  # Clear LCD

    display.set_cursor_pos(0, 0)  # First row
    display.print(line1)  # Print first line

    display.set_cursor_pos(1, 0)  # Second row
    display.print(line2)  # Print second line

# ============================================================
# LED FUNCTION
# ============================================================

def update_leds():  # Update LEDs status

    slot1_led.value = parking_slots[0]  # Slot 1 LED state
    slot2_led.value = parking_slots[1]  # Slot 2 LED state
    slot3_led.value = parking_slots[2]  # Slot 3 LED state

    if occupied_slots < TOTAL_SLOTS:  # If parking not full

        green_led.value = True  # Green ON
        red_led.value = False  # Red OFF

    else:  # If parking full

        green_led.value = False  # Green OFF
        red_led.value = True  # Red ON

# ============================================================
```

```
# KEYPAD FUNCTION
# ============================================================

def read_key():  # Read keypad button

    if data_available.value:  # If key pressed

        value = 0  # Start value

        for i, pin in enumerate(data_inputs):  # Read keypad bits

            if pin.value:  # If bit is HIGH

                value |= (1 << (3 - i))  # Build binary value

        while data_available.value:  # Wait until key release
            pass

        return keypad_map.get(value, '?')  # Return keypad value

    return None  # No key pressed

# ============================================================
# ENTRY LIGHT FUNCTION
# ============================================================

def entry_open():  # Open entry gate

    entry_lamp.value = True  # Turn on entry lamp
    alarm_led.value = True  # Turn on alarm LED

    time.sleep(0.3)  # Delay

    alarm_led.value = False  # Turn off alarm LED

    time.sleep(3)  # Keep gate open

    entry_lamp.value = False  # Turn off entry lamp

# ============================================================
# EXIT LIGHT FUNCTION
# ============================================================

def exit_open():  # Open exit gate

    exit_lamp.value = True  # Turn on exit lamp
    alarm_led.value = True  # Turn on alarm LED

    time.sleep(0.3)  # Delay

    alarm_led.value = False  # Turn off alarm LED

    time.sleep(3)  # Keep gate open

    exit_lamp.value = False  # Turn off exit lamp

# ============================================================
# STARTUP
# ============================================================

entry_lamp.value = False  # Entry lamp OFF
exit_lamp.value = False  # Exit lamp OFF

update_display()  # Update display
update_leds()  # Update LEDs

```

```
show_message("PARKING LOCKED", "ENTER PASSWORD")  # Startup message

# ========================================================
# MAIN LOOP
# ========================================================

while True:  # Infinite loop

    update_display()  # Refresh display
    update_leds()  # Refresh LEDs

    key = read_key()  # Read keypad input

    # ======================================================
    # PASSWORD SYSTEM
    # ======================================================

    if system_open == False:  # If system locked

        if key:  # If key pressed

            if key == "#":  # Confirm button

                if user_input == password:  # Correct password

                    system_open = True  # Open system

                    attempts = 3  # Reset attempts

                    user_input = ""  # Clear input

                    show_message("ACCESS GRANTED", "PARKING OPEN")  # Success message

                    green_led.value = True  # Green LED ON
                    alarm_led.value = True  # Alarm LED ON

                    time.sleep(1)  # Delay

                    alarm_led.value = False  # Alarm LED OFF

                    show_slots()  # Show slots

                else:  # Wrong password

                    attempts -= 1  # Reduce attempts

                    show_message("WRONG PASSWORD", "TRY AGAIN")  # Error message

                    red_led.value = True  # Red LED ON
                    alarm_led.value = True  # Alarm LED ON

                    time.sleep(1)  # Delay

                    red_led.value = False  # Red LED OFF
                    alarm_led.value = False  # Alarm LED OFF

                    if attempts == 0:  # If no attempts left

                        show_message("SYSTEM LOCKED", "WAIT 10 SEC")  # Lock message

                        for seconds in range(10, -1, -1):  # Countdown loop

                            segment.send_bcd(seconds % 10)  # Show countdown

                            s71.value = False  # Activate latch
                            time.sleep(0.001)  # Small delay
```

```
                            s71.value = True  # Disable latch

                            red_led.value = True  # Red LED ON
                            green_led.value = True  # Green LED ON

                            slot1_led.value = True  # Slot 1 LED ON
                            slot2_led.value = True  # Slot 2 LED ON
                            slot3_led.value = True  # Slot 3 LED ON

                            alarm_led.value = True  # Alarm LED ON

                            time.sleep(0.5)  # Delay

                            red_led.value = False  # Red LED OFF
                            green_led.value = False  # Green LED OFF

                            slot1_led.value = False  # Slot 1 LED OFF
                            slot2_led.value = False  # Slot 2 LED OFF
                            slot3_led.value = False  # Slot 3 LED OFF

                            alarm_led.value = False  # Alarm LED OFF

                            time.sleep(0.5)  # Delay

                        attempts = 3  # Reset attempts

                    user_input = ""  # Clear input

                    show_message("PARKING LOCKED", "ENTER PASSWORD")  # Lock message

            elif key == "*":  # Delete button

                user_input = user_input[:-1]  # Remove last digit

            elif len(user_input) < 4:  # Limit password length

                if key.isdigit():  # Accept numbers only

                    user_input += key  # Add digit

            if system_open == False:  # If still locked

                display.set_cursor_pos(1, 0)  # Second row
                display.print("                ")  # Clear row

                display.set_cursor_pos(1, 0)  # Second row
                display.print("*" * len(user_input))  # Show hidden password

    # ====================================================
    # PARKING SYSTEM
    # ====================================================

    else:  # If system open

        # ================================================
        # EXIT BUTTON
        # ================================================

        if not exit_button.value:  # If exit button pressed

            show_message("CAR EXITED", "EXIT OPENING")  # Exit message

            exit_open()  # Open exit gate

            occupied_slots = max(0, occupied_slots - 1)  # Reduce occupied slots

```

```
453.             for i in range(3):  # Loop through slots
454.
455.                 if parking_slots[i] == 1:  # Find occupied slot
456.
457.                     parking_slots[i] = 0  # Free slot
458.                     break  # Exit loop
459.
460.             update_leds()  # Update LEDs
461.
462.             show_slots()  # Show slot status
463.
464.             time.sleep(1)  # Delay
465.
466.         # ================================================
467.         # ALARM BUTTON
468.         # ================================================
469.
470.         if not alarm_button.value:  # If alarm button pressed
471.
472.             show_message("EMERGENCY!", "ALARM ACTIVE")  # Alarm message
473.
474.             for i in range(15):  # Alarm flashing loop
475.
476.                 red_led.value = True  # Red LED ON
477.                 green_led.value = True  # Green LED ON
478.
479.                 slot1_led.value = True  # Slot 1 LED ON
480.                 slot2_led.value = True  # Slot 2 LED ON
481.                 slot3_led.value = True  # Slot 3 LED ON
482.
483.                 alarm_led.value = True  # Alarm LED ON
484.
485.                 time.sleep(0.15)  # Delay
486.
487.                 red_led.value = False  # Red LED OFF
488.                 green_led.value = False  # Green LED OFF
489.
490.                 slot1_led.value = False  # Slot 1 LED OFF
491.                 slot2_led.value = False  # Slot 2 LED OFF
492.                 slot3_led.value = False  # Slot 3 LED OFF
493.
494.                 alarm_led.value = False  # Alarm LED OFF
495.
496.                 time.sleep(0.15)  # Delay
497.
498.             update_leds()  # Update LEDs
499.
500.             show_slots()  # Show slots
501.
502.         # ================================================
503.         # SLOT SELECTION
504.         # ================================================
505.
506.         if key in ["1", "2", "3"]:  # If slot selected
507.
508.             slot = int(key) - 1  # Convert slot number
509.
510.             if parking_slots[slot] == 0:  # If slot free
511.
512.                 show_message("SLOT " + key + " FREE", "ENTRY OPENING")  # Free message
513.
514.                 entry_open()  # Open entry gate
515.
516.                 parking_slots[slot] = 1  # Mark slot occupied
517.
518.                 occupied_slots += 1  # Increase occupied count
```

```
519.
520.             else:  # If slot busy
521.
522.                 show_message("SLOT " + key + " BUSY", "CHOOSE ANOTHER")  # Busy
message
523.
524.                 time.sleep(1)  # Delay
525.
526.             update_leds()  # Update LEDs
527.
528.             show_slots()  # Show slots
529.
530.     time.sleep(0.1)  # Small loop delay
531.
```

APPENDIX B: ADDITIONAL HARDWARE SCREENSHOTS

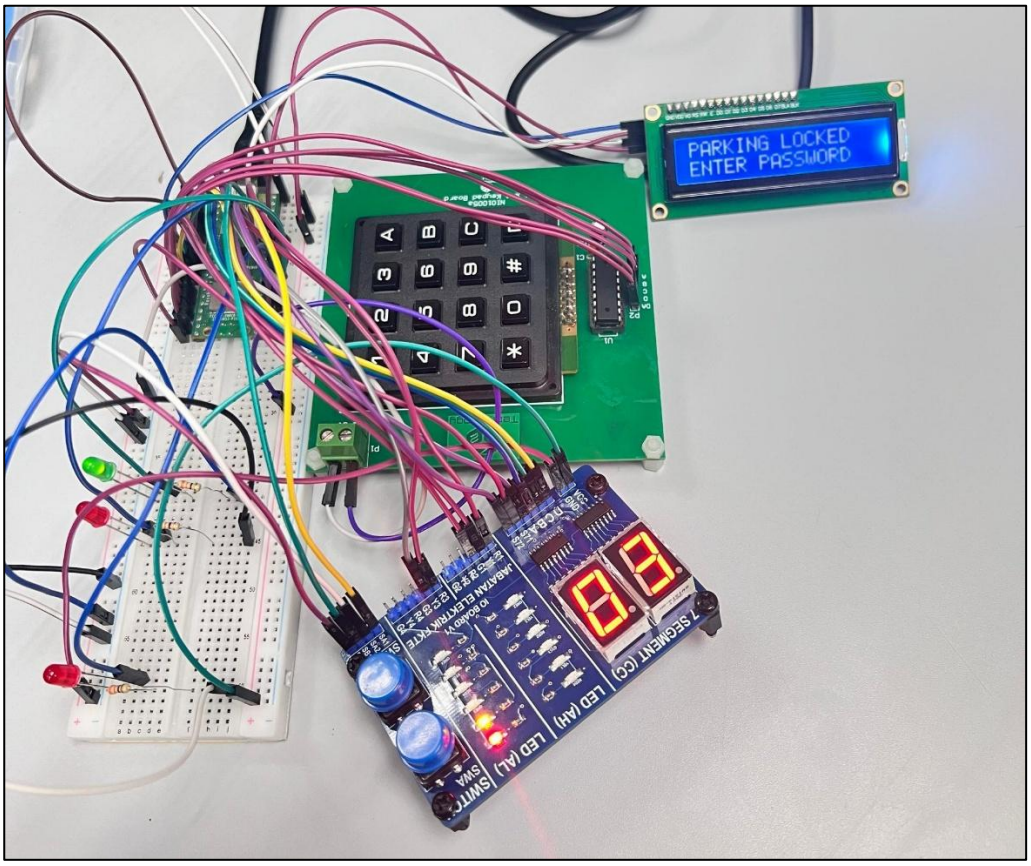

Figure 8: Hardware Startup and Password Authentication Mode.

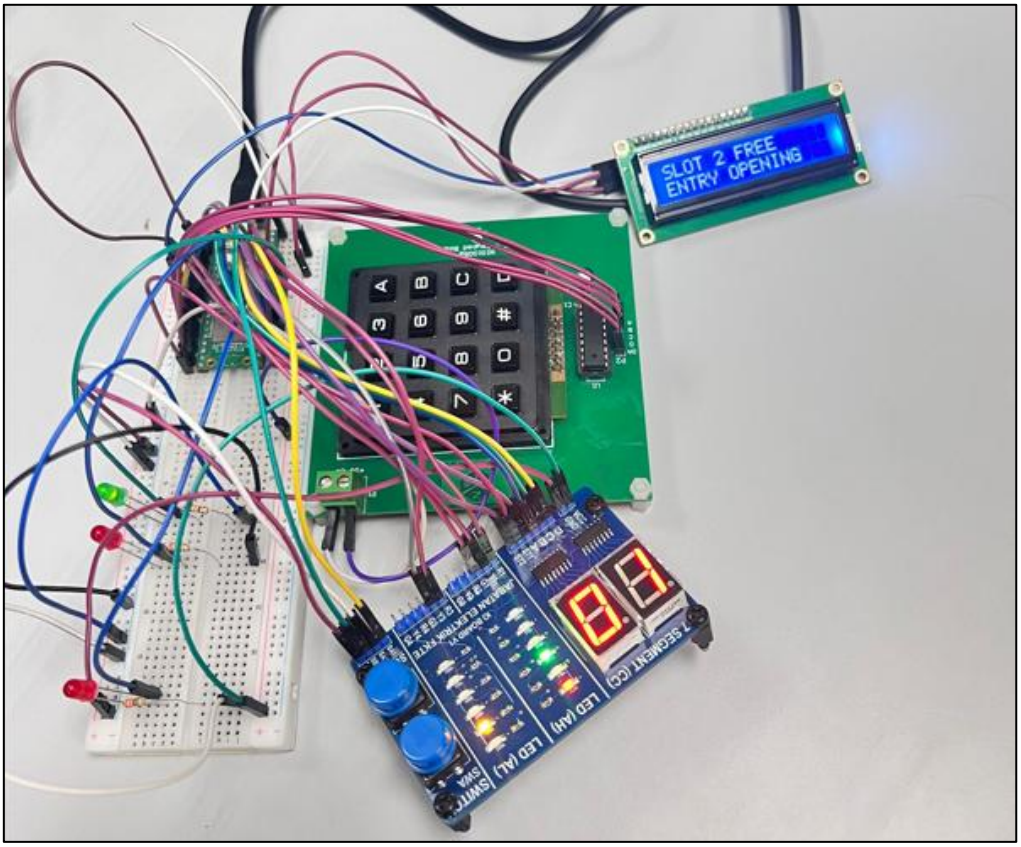

Figure 9: Hardware Parking Slot Monitoring Operation.

Appendix C: Additional Wokwi Simulation Screenshots

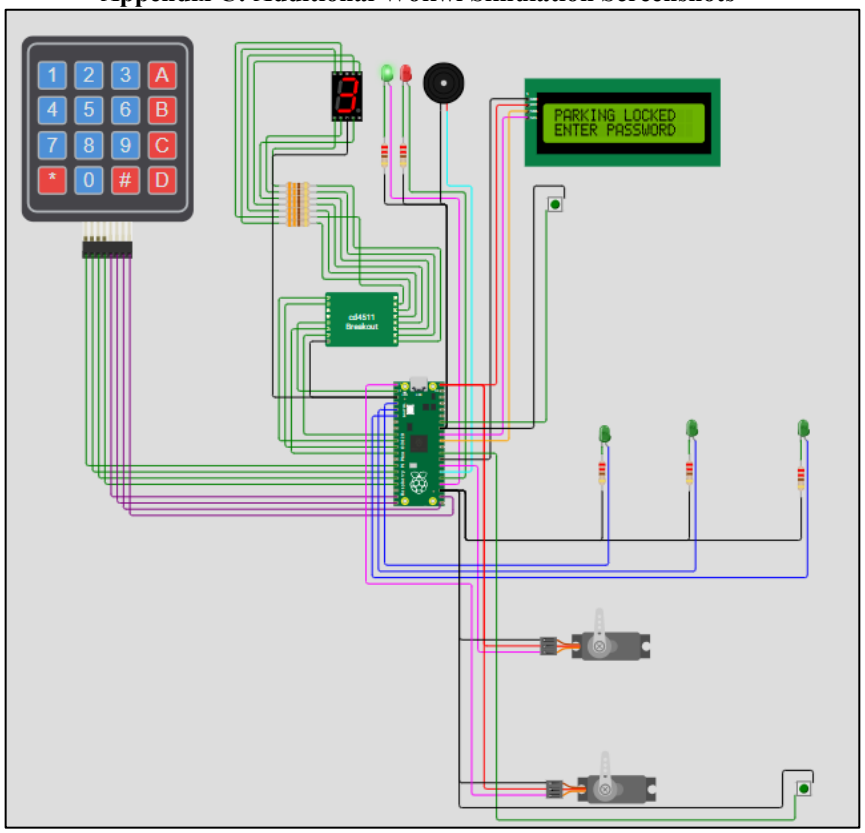

Figure 10: Wokwi Simulation Startup Mode.

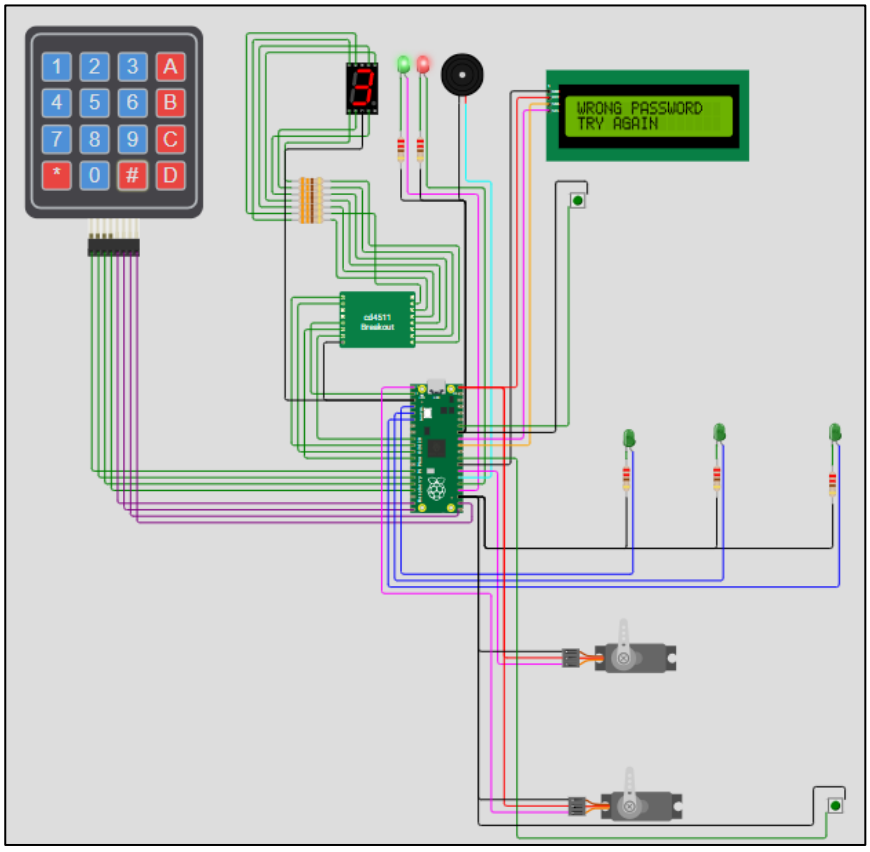

Figure 11: Wokwi Simulation Wrong Password Detection.

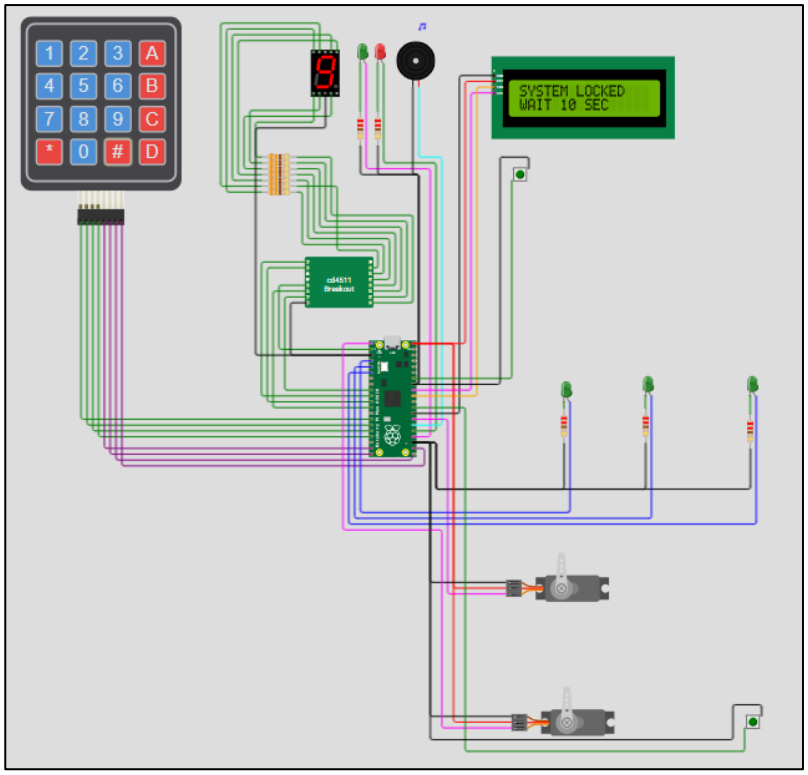

Figure 12: Wokwi Simulation Temporary Lock Condition.

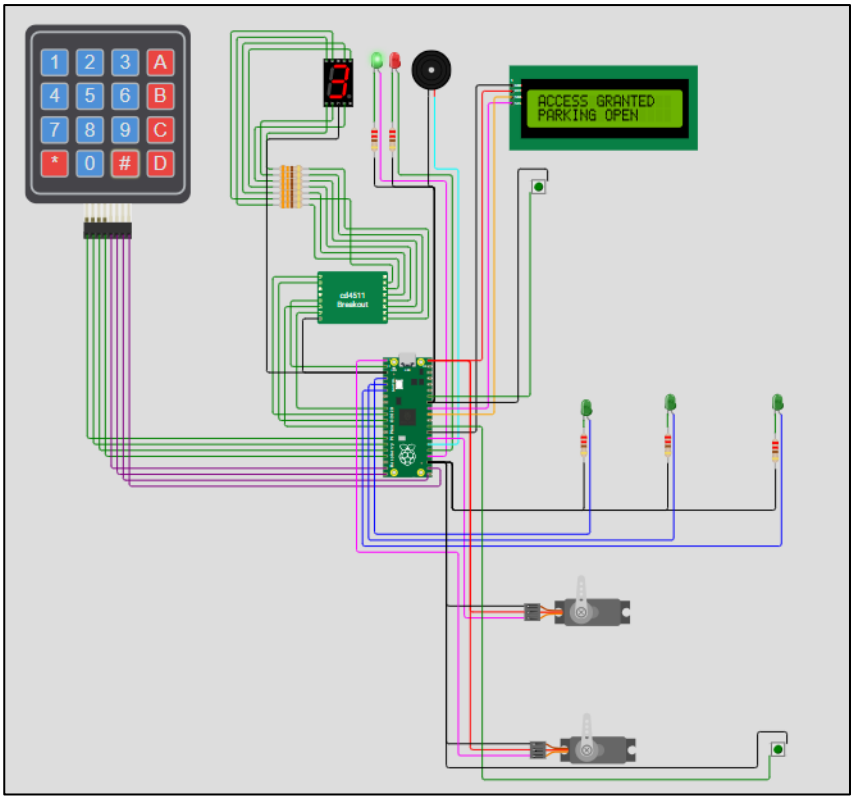

Figure 13: Wokwi Simulation Access Granted Operation.

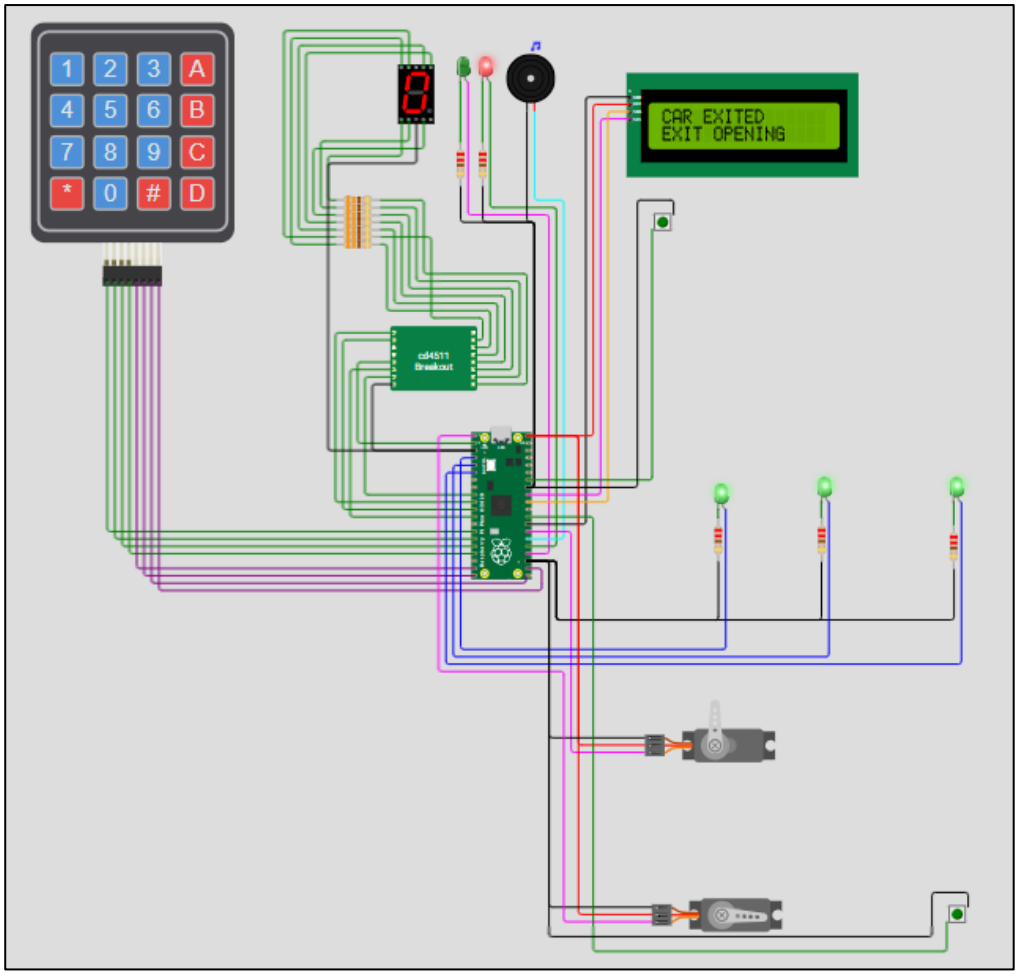

Figure 14: Wokwi Simulation Vehicle Exit Operation.

APPENDIX D: DATASHEETS

The following datasheets and technical references were used during the design, simulation, and hardware implementation of the proposed Smart Parking System.

- Raspberry Pi Pico W Microcontroller Datasheet: https://pip-assets.raspberrypi.com/categories/686-raspberry-pi-pico-w/documents/RP-008312-DS-1-pico-w-datasheet.pdf
- MM74C922 Keypad Encoder Datasheet: https://www.mouser.com/datasheet/2/308/MM74C922-1120961.pdf
- CD4511 BCD to 7-Segment Decoder Datasheet: https://www.ti.com/lit/ds/symlink/cd74hct4511.pdf
- 16×2 I2C LCD Display Datasheet: https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf
- 7-Segment Display Datasheet: https://mil.ufl.edu/3701/docs/OOTB_Max10/sevenSegmentDatasheet.pdf
- Push Button Switch Datasheet: https://www.handsontec.com/dataspecs/switches/Tact%20Switch%206x6.pdf
- LED Datasheet: https://ieee.ics.uci.edu/ops/assets/datasheets/ops_all_leds_datasheet.pdf

These datasheets were used to verify component specifications, operating voltage, GPIO interfacing requirements, decoder operation, LCD communication methods, and hardware compatibility during both simulation and hardware implementation stages.

APPENDIX E: INDIVIDUAL TASK MAPPING

Table 5: Individual Task Mapping.

No.	Member Name	Matric ID	Assigned Simulation or Hardware Task	Report Section Task
1.	Adel Husham Mohamedain Yousuf	2301090057-1	System programming, hardware integration, simulation development, and system testing	Introduction, Methodology, Results, Discussion & Conclusion
2.	Raudhah Binti Abdul Rahim	231093650	LCD display interfacing, keypad encoder configuration, and hardware assembly	Methodology & Hardware Description
3.	Nurul Nadhirah Alya Binti Mohamad Nizam	231092874	Wokwi simulation setup, component configuration, and simulation verification	Results & Simulation Analysis
4.	Muhammad Nadzmi Bin Kamarulzaman	221093742	Hardware troubleshooting, circuit verification, documentation formatting, references, and appendix preparation	Discussion, References & Appendix